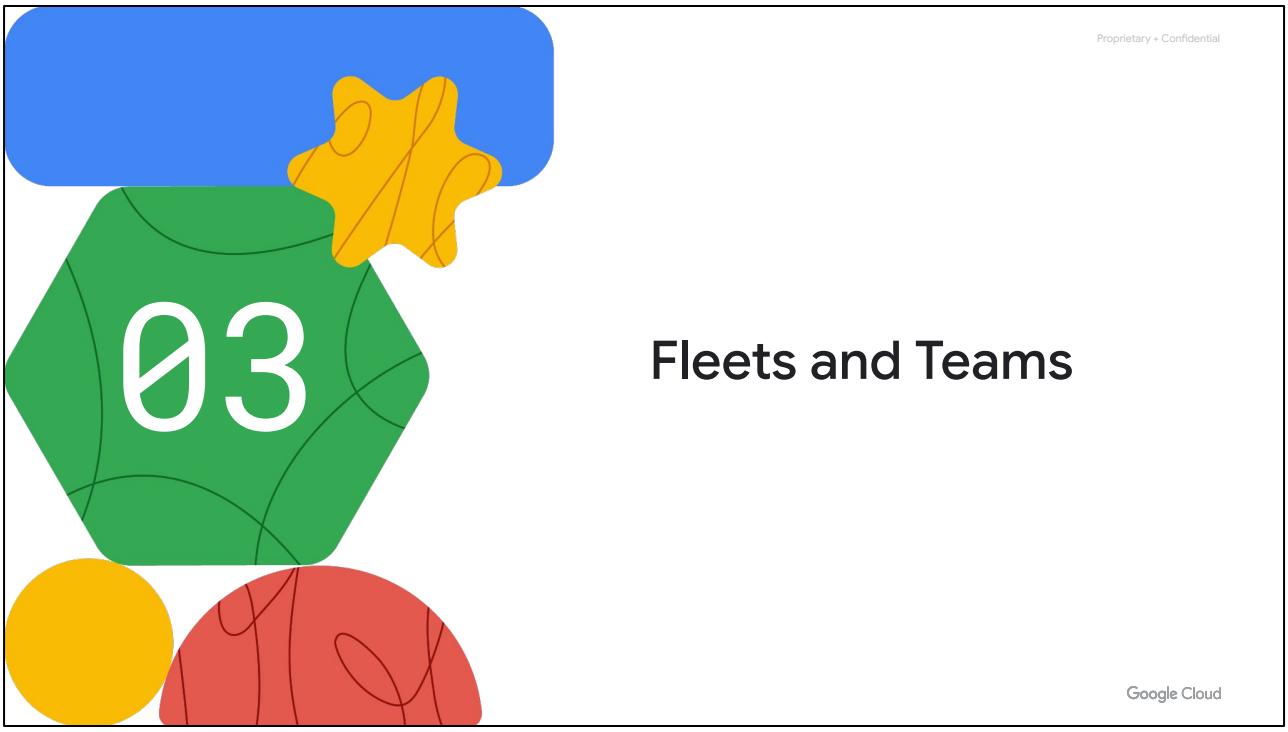


Small-scale organizations can often get away with using a single cluster. But as an organization grows and embraces cloud-native technologies like containers and service meshes, that single cluster may no longer be sufficient.

Running multiple clusters has several advantages, like workload distribution and separation of development and production environments.

As you add more clusters, management and governance becomes increasingly difficult. Many organizations build their own custom tooling and operational policies to meet these demands.

But there's an alternative to this solution: fleets in GKE.

Proprietary + Confidential

Fleets and Teams

01 GKE fleets

02 Example fleet solutions

03 Fleet team management

04 Fleet management

Google Cloud

This section is about fleets and teams in GKE. You'll learn to:

- Define GKE fleets.
- Describe how GKE fleets can solve common cluster management problems.
- Manage fleets and teams in GKE.
- Detail the elements of Fleet management.

Proprietary + Confidential

Fleets and Teams

01 GKE fleets

02 Example fleet solutions

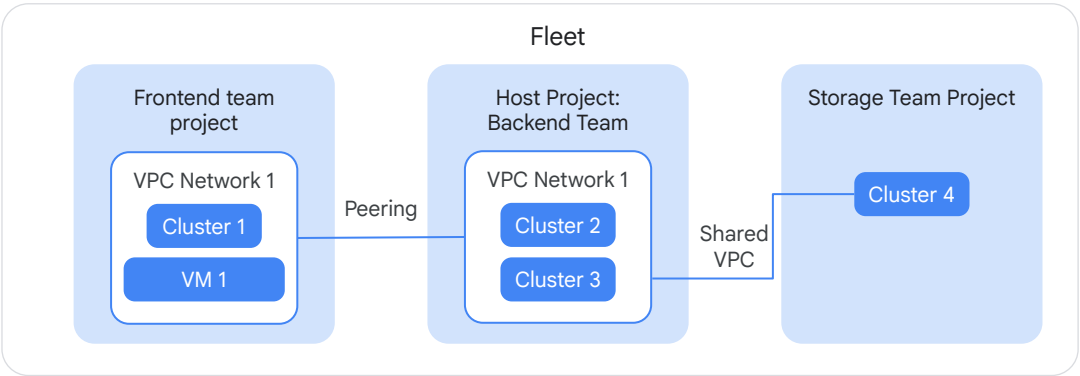
03 Fleet team management

04 Fleet management

So, what are fleets in GKE and how can they be used to simplify administration and management?

Proprietary + Confidential

What are fleets?



Fleets are logical groupings of Kubernetes clusters with consistent configurations and policies.

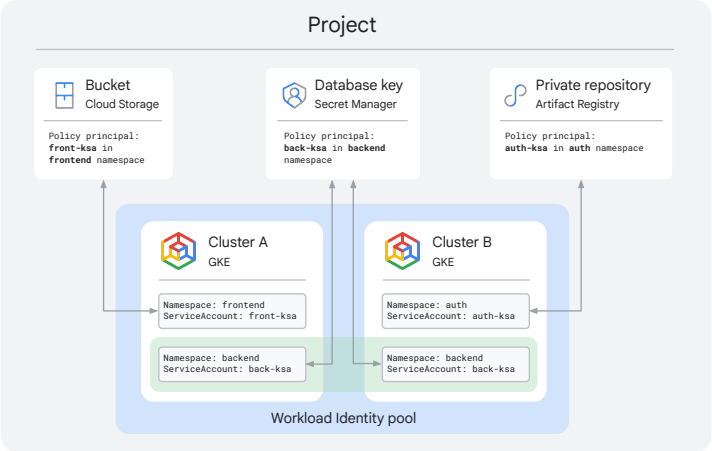
Google Cloud

Fleets are logical groupings of Kubernetes clusters with consistent configurations and policies.

Created within a host project, fleets consist of GKE clusters, and a selection of fleet-enabled GKE components spanning across Google Cloud VPC networks and projects.

Proprietary + Confidential

Identity Sameness



Google Cloud

GKE clusters and other fleet-enabled components simplify cluster and service management by leveraging fleet concepts like namespace and identity sameness.

Proprietary + Confidential

Fleet-enabled components

Workload identity pools	Simplify service authentication and authorization.
Multi-cluster gateways	Ensure low-latency and high-availability.
Cloud Service Mesh	Monitor and manage a reliable service mesh on Google Cloud.
Configuration and policy management	Deploy, monitor, and enforce declarative policies from a central Git repository.

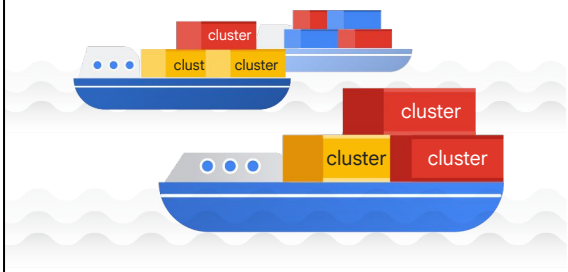
Google Cloud

Examples of fleet enabled-components include:

- Workload identity pools for GKE clusters. These pools simplify service authentication and authorization within a service mesh and outside the cluster.
- Multi-cluster gateways ensure low-latency and high-availability by defining load balancing endpoints for clusters and services.
- Cloud Service Mesh is used to monitor and manage a reliable service mesh on Google Cloud, on-premises, or on other supported cloud providers.
- Configuration and policy management simplifies configuration management. It deploys, monitors, and enforces declarative policies from a central Git repository.

Proprietary + Confidential

Fleet benefits



- Consistent namespaces, services, workload identities, mesh identity, and access.
- Each fleet-aware resource (like a namespace or service) belongs to only one fleet.
- Security policies applied at the fleet level automatically extend to all clusters within it.
- Frequently communicating resources can be placed in the same fleet.
- Flexible service organization for easier administration.
- Centralize policy and governance while retaining multi-cluster benefits.

Google Cloud

Let’s explore the benefits of using fleets to simplify the management of multiple Kubernetes clusters.

- Fleets ensure all clusters have the same namespaces, services, workload identities, mesh identity, and access.
- Each fleet-aware resource (like a namespace or service) belongs to only one fleet. This "single source of truth" eliminates conflicts and simplifies management.
- Fleets establish a trust domain. Security policies applied at the fleet level automatically extend to all clusters within it.
- Resources that need to communicate frequently can be placed in the same fleet for optimal performance.
- Organize services by environment, line of business, or deployment schedule for easier administration.
- Retain the benefits of multiple clusters while centralizing policy and governance at the fleet level.

Proprietary + Confidential

Create a Fleet

```
gcloud container fleet create \
--display-name=gke-fleet
```

Use gcloud or the Google Cloud console to create a fleet.

Google Cloud

Before wrapping up, let's explore how to create a fleet, and register a new or existing cluster to a fleet.

To create a fleet, use the gcloud container fleet create command or the Google Cloud console to create a fleet.

Proprietary + Confidential

Register a new cluster to a fleet

```
gcloud container clusters \
create-auto my-auto-cluster
--enable-fleet
```

Create and register a new GKE Autopilot cluster.

```
gcloud container clusters create \
my-standard-cluster --enable-fleet \
--workload-pool=myproject.svc.id.goog
```

Create and register a new GKE standard cluster.

Google Cloud

After you create a fleet, you can register a new or existing cluster to it. To create and register a new GKE Autopilot cluster, add the --enable-fleet flag to the gcloud container clusters create-auto command.

To create and register a new GKE standard cluster, enable the fleet and Workload Identity Federation.

Proprietary + Confidential

Register an existing cluster to a fleet

```
gcloud container clusters update \
my-auto-cluster --enable-fleet
```

Add an existing cluster to the project fleet.

```
gcloud container clusters update \
my-standard-cluster \
--fleet-project=myproject
```

Register the cluster to a fleet in a different project.

Google Cloud

Use the update command with the --enable-fleet flag to add an existing cluster to the project fleet. To register a cluster to a fleet in a different project, use the --enable-project flag.

Proprietary + Confidential

Fleets and Teams

01 GKE fleets

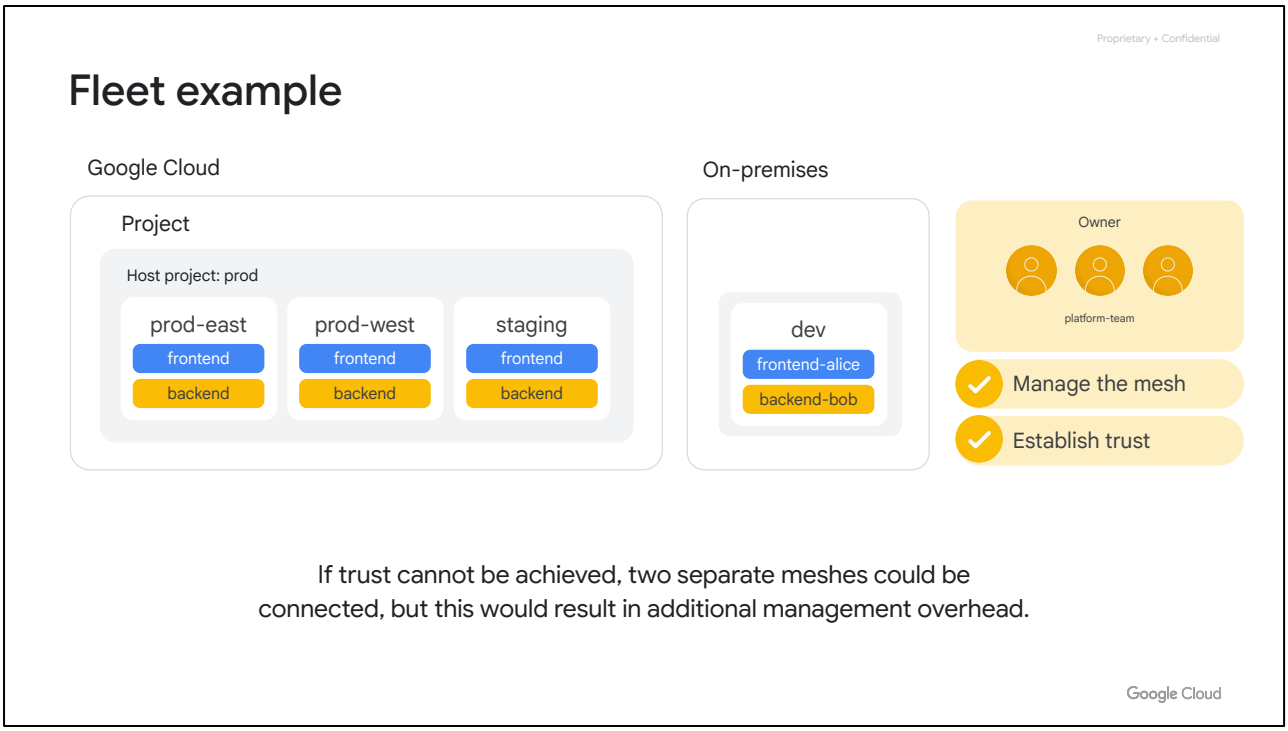
02 Example fleet solutions

03 Fleet team management

04 Fleet management

Google Cloud

GKE fleets are useful for grouping, sameness, exclusivity, and trust. Let's explore a few examples of how fleets provide a management solution for complex Kubernetes environments.



Let's say there are four clusters: two for production (in two regions for redundancy), one for staging and testing, and one for development. The development environment is hosted on premises.

A platform team centrally owns and administers all four clusters. There are also two services: frontend and backend. More complex scenarios may include more services and clusters.

The platform team is responsible for managing the mesh and establishing trust between the frontend and backend teams.

If trust cannot be achieved, two separate meshes could be connected, but this would result in additional management overhead.

Proprietary + Confidential

Isolation vs. consistency

Google Cloud

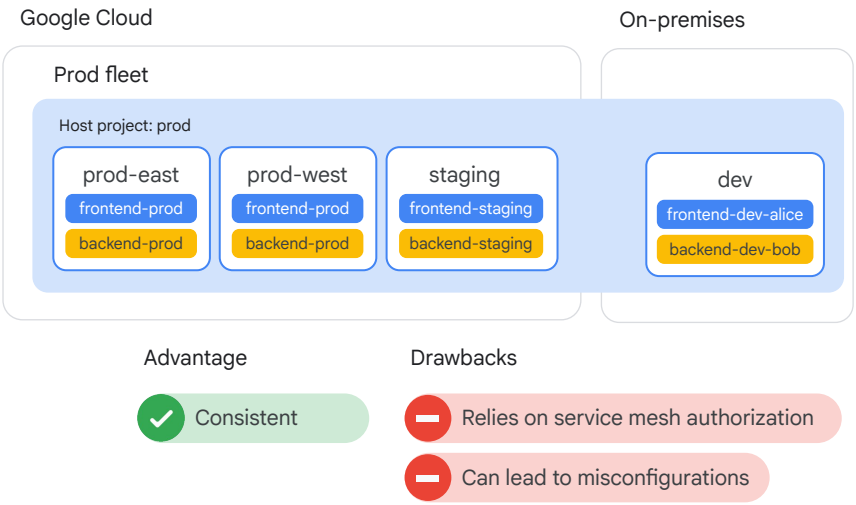
When considering possible implementations, it's important to determine what should be prioritized: *isolation* between different resource types or *consistency* across them.

More consistency is easier to achieve with fewer fleets.

Let's examine three potential fleet implementations.

Proprietary + Confidential

Fleet solution #2



Advantage

- Consistent

Drawbacks

- Relies on service mesh authorization
- Can lead to misconfigurations

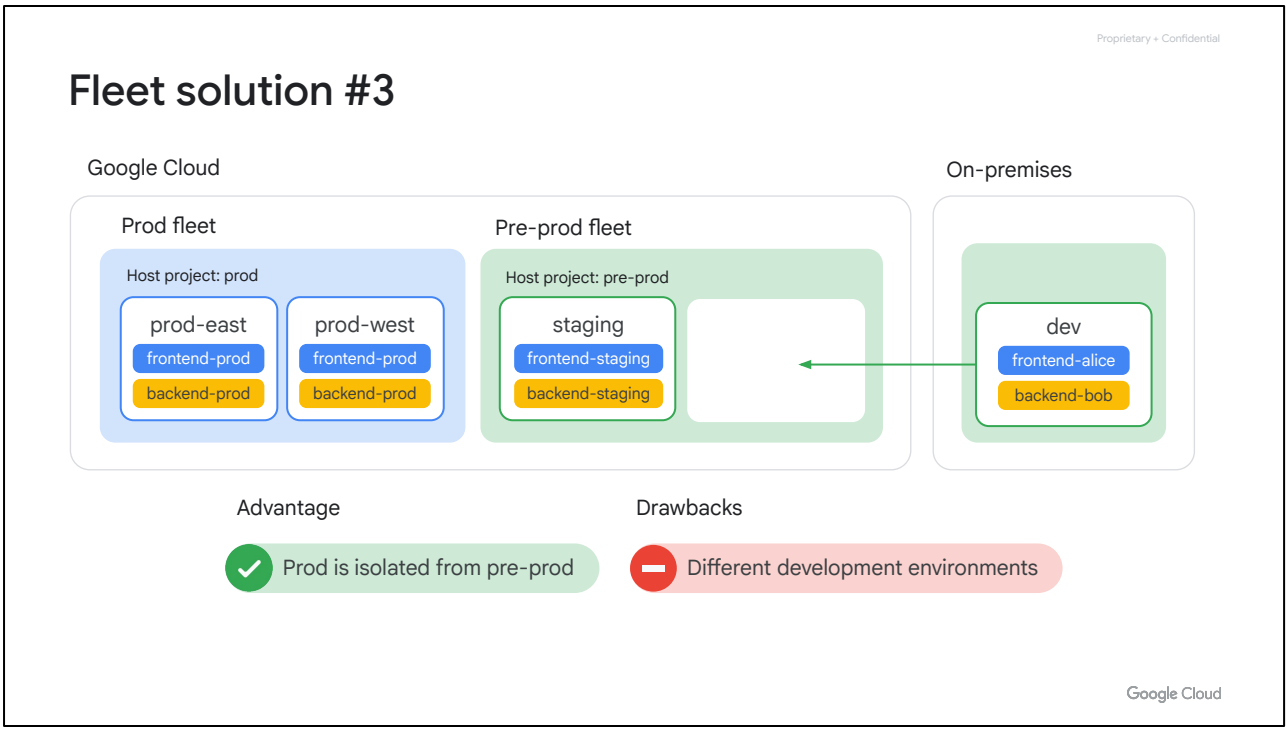
Google Cloud

Another option is to keep your resources in the example project and create the fleet there. With this approach, namespaces and services must be normalized throughout the fleet, making it easy to achieve consistency across environments.

In this example, the generic frontend is renamed to frontend-prod and frontend-staging in the production and staging clusters. While you could keep the original names, it's helpful to rename your namespaces to indicate that they are for development. Like frontend-dev-alice in this example.

This approach trades isolation for consistency and ease of management. Cloud Service Mesh is used for authorization and preventing unwanted communication, which can be cumbersome and prone to misconfigurations.

You can also apply policies across all resources to ensure that development closely reflects production.



The final approach we'll explore combines the staging and development resources into a non-production fleet and places production resources in a separate fleet. This option represents the middle ground between isolation and consistency.

Resources for production are placed in the production fleet host project, and the on-premises development cluster is registered into the non-production fleet.

For clarity, namespaces and services can be normalized between the staging and development resources. For example, frontend is renamed to frontend-staging in the staging cluster.

The advantage of this solution is that production and non-production are isolated from each other. This allows developers to test their services against staged versions. For example, Alice can connect her frontend service to a staged backend during development. The disadvantage is that the environments are different.

Proprietary + Confidential

Fleets and Teams

01 GKE fleets

02 Example fleet solutions

03 Fleet team management

04 Fleet management

Google Cloud

Managing Kubernetes infrastructure for multiple teams can be complex. GKE fleet team management provides platform admins with tools and flexibility to provision and manage infrastructure resources for multiple teams.

Fleet team management introduces the concept of team-level abstraction, where admins define team scopes and fleet namespaces for efficient onboarding, access control, and resource optimization.

Proprietary + Confidential

What are scopes?

- Enable operators and admins to access information like resource utilization, logs, errors, and metrics.
- Can help facilitate resource assessment, troubleshooting, and rollout sequencing upgrades.
- Members can use Connect gateway to login with their Google credentials and authenticate with Google Groups.

Let's define scopes in GKE. With team scopes, operators and admins can access information defined by the scope, like resource utilization, logs, errors, and metrics.

Team scopes can help facilitate resource assessment, troubleshooting, and rollout sequencing upgrades.

To access their team scope clusters, members can use Connect gateway to login with their Google credentials and authenticate with Google Groups.

Proprietary + Confidential

What are namespaces?

- Enable operators and admins to access resources defined by the namespace, like deployments, services, and pods.
- Can help facilitate resource isolation, access control, and application lifecycle management.
- Members can use can use kubectl with configured context and authenticate with Kubernetes RBAC.

Google Cloud

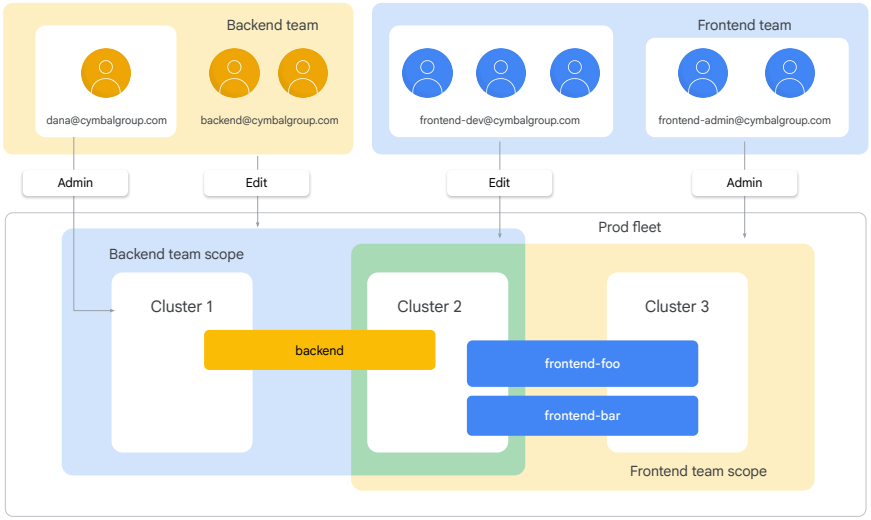
Next, let's define namespaces in GKE. With namespaces, operators and developers can access resources defined by the namespace, like deployments, services, and pods.

Namespaces can help facilitate resource isolation, access control, and application lifecycle management.

To access their namespace resources, users can use `kubectl` with configured context and authenticate with Kubernetes RBAC.

Proprietary + Confidential

Example of a team structure in GKE



Google Cloud

So how does this work? Let's say, for example, an organization has two primary teams. One for the backend and one for the frontend. The admin can create separate team scopes, each associated with specific clusters and namespaces.

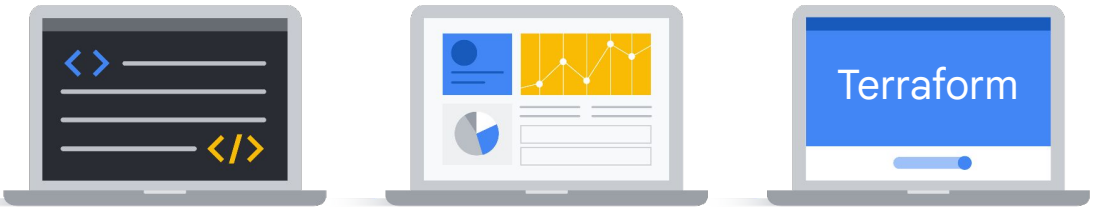
The backend team has access to clusters 1 and 2 with the namespace "backend," while the frontend team has access to clusters 2 and 3 with namespaces "frontend-foo" and "frontend-bar."

Both teams share cluster 2 while maintaining their isolated namespaces.

Team leads are granted admin access, allowing them to manage roles and bindings within their scope, while other team members have editor access.

Proprietary + Confidential

There are three ways to configure fleet teams



gcloud commands

Google Cloud console

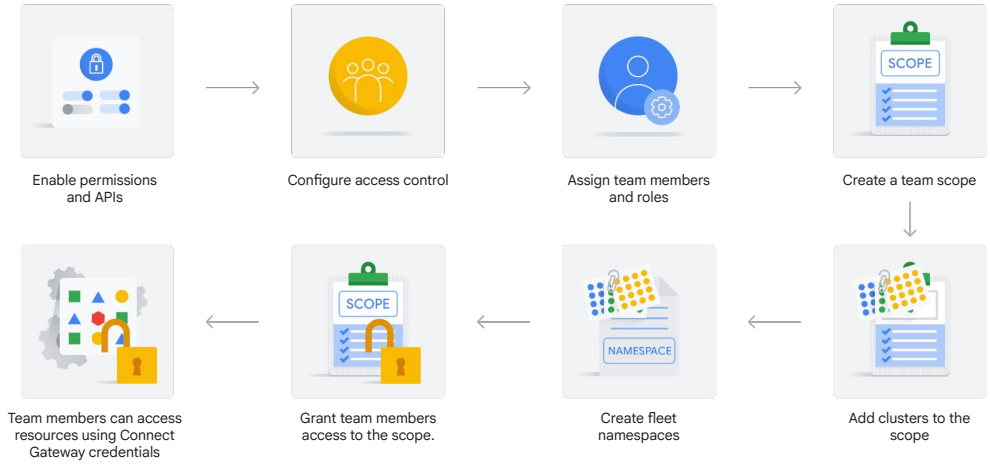
Terraform

Google Cloud

Platform admins can create and configure fleet teams by using gcloud commands, the Google Cloud console, or Terraform. Let's explore the general steps to create and configure teams.

Proprietary + Confidential

Create and configure fleet teams



Google Cloud

First, enable the necessary permissions and APIs for your fleet.

Then set up access control using Google Groups. This is the recommended approach.

To control team access to fleet resources, assign team members and roles. Each member can be assigned different access levels: Scope Admin, Scope Editor, or Scope Viewer.

Create a team scope and give it a unique name.

Then add your clusters to the team scope. Make sure each cluster is already part of your fleet and uses Google Groups for access control.

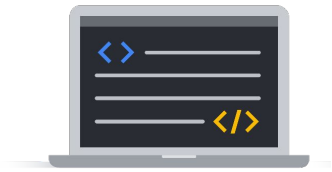
Now create fleet namespaces within the team scope. This gives you granular control over resources and isolates each team's work within the fleet.

Grant team members access to the scope using their Google Groups or individual accounts. This streamlines the process of onboarding teams by providing pre-defined access controls and resource allocation.

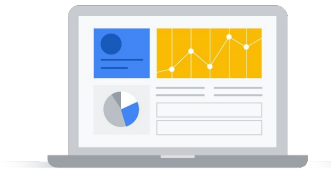
Team members can then access their resources using Connect gateway credentials.

Proprietary + Confidential

Managing team scopes and namespaces



Command Line Interface



Google Cloud console

- List, update, and delete scopes
- Add or remove clusters from scopes
- Create and delete fleet namespaces
- Manage team member access

Google Cloud

Admins can manage team scopes and fleet namespaces using the Google Cloud command line interface or the Google Cloud console. Management tasks include listing, updating, and deleting scopes, adding or removing clusters from scopes, creating and deleting fleet namespaces, and managing team member access.

Proprietary + Confidential

Managing team scopes

```
gcloud container fleet scopes create \
SCOPE_NAME
```

Create a team scope

```
gcloud container fleet memberships \
bindings create BINDING_NAME \
--membership MEMBERSHIP_NAME \
--scope  SCOPE_NAME \
--location MEMBERSHIP_LOCATION
```

Add clusters to a team scope

```
gcloud container fleet scopes \
namespaces create NAMESPACE_NAME \
--scope=SCOPE_NAME
```

Create fleet namespaces

Google Cloud

Let's explore common commands used to manage team scopes.

The `gcloud container fleet scopes` create command creates a team scope.

To add clusters to a team scope, use the `gcloud container fleet memberships bindings create` command.

To create fleet namespaces, use the `gcloud container fleet scopes namespaces create` command.

Proprietary + Confidential

Managing team scopes

```
gcloud container fleet scopes \
add-app-operator-binding SCOPE_ID \
--role=ROLE --group=TEAM_EMAIL
```

Grant team members access to the team scope

```
gcloud container fleet memberships \
bindings delete BINDING_NAME \
--membership MEMBERSHIP_NAME
```

Remove clusters from a team scope

```
gcloud container fleet scopes \
remove-app-operator-binding SCOPE_NAME \
--group=TEAM_EMAIL
```

Remove team members from team scope

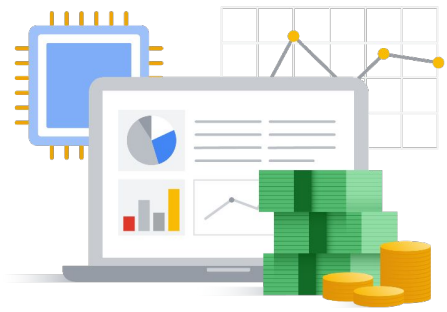
Google Cloud

The `gcloud container fleet scopes add-app-operator-binding` command grants team members access to the team scope.

Use the `gcloud container fleet memberships bindings delete` to remove clusters from a team scope, and the `gcloud container fleet scopes remove` command to remove team members from a team scope.

Proprietary + Confidential

Monitoring and resource utilization



Google Cloud console > Teams > Monitoring

- CPU, memory, and disk usage
- Error and restart counts
- Estimated costs

Google Cloud

Within the Teams page in the Google Cloud console, the Monitoring tab displays team scope resource utilization details, like CPU, memory and disk usage, error and restart counts, and estimated costs. This information can be useful for resource allocation and troubleshooting.

Proprietary + Confidential

Fleets and Teams

01 GKE fleets

02 Example fleet solutions

03 Fleet team management

04 Fleet management

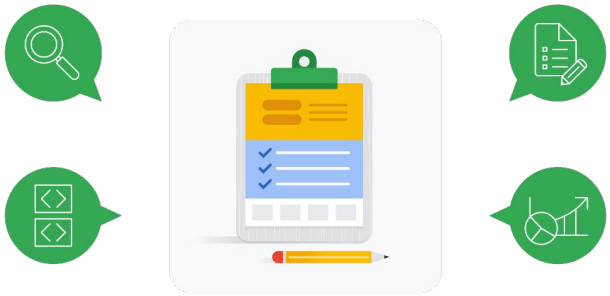
Google Cloud

Part of managing containerized workloads is maintaining updates to the underlying software.

But how is this done at scale?

Proprietary + Confidential

The fleet host project is the control center for fleet management



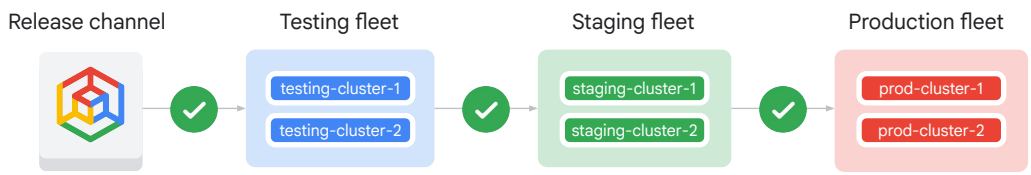
Fleet Host Project

Google Cloud

Fleets provide a control plane and unified view across diverse environments simplifying multi-cluster management. In GKE, the Fleet Host Project acts as the control center for fleet management and provides a centralized interface for monitoring and configuration.

Proprietary + Confidential

Rollout sequencing is used to manage software updates at scale



Controlled and phased upgrades of GKE clusters across multiple environments

Test and validate in pre-production environments before deploying upgrades to production

Google Cloud

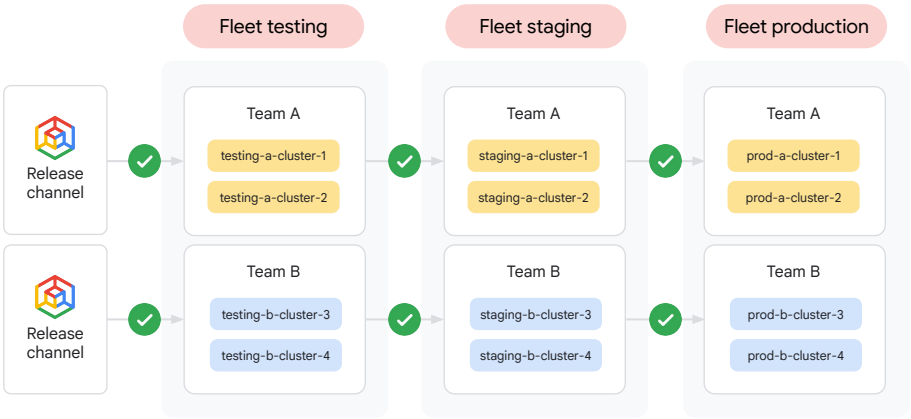
Rollout sequencing is a fleet management feature used to manage software updates at scale.

Rollout sequencing enables controlled and phased upgrades of GKE clusters across multiple environments. To minimize risk and ensure application stability, developers can test and validate in pre-production environments before deploying upgrades to production.

You can group clusters based on deployment environments like testing, staging, and production, or register each cluster with a fleet based on your desired grouping.

Proprietary + Confidential

Clusters are organized into groups

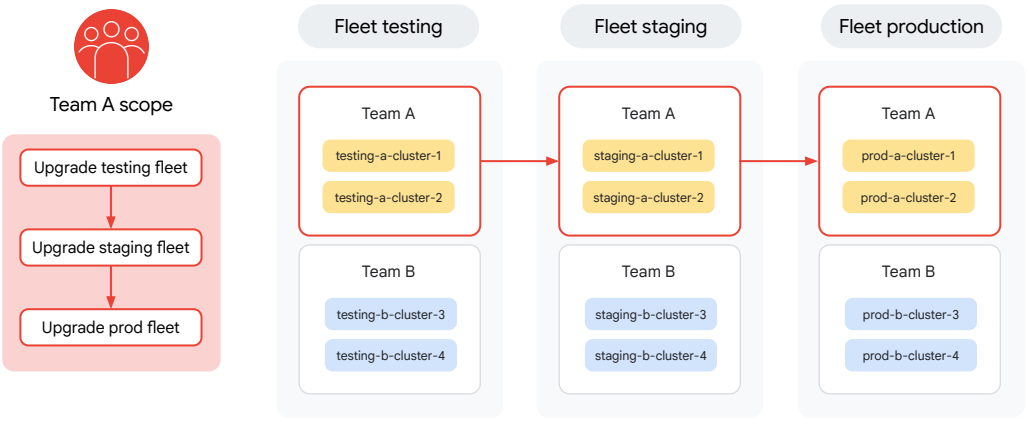


Google Cloud

Clusters are organized into **groups** based on release channels and minor versions. These groups can be fleets or team scopes.

Proprietary + Confidential

Team scopes



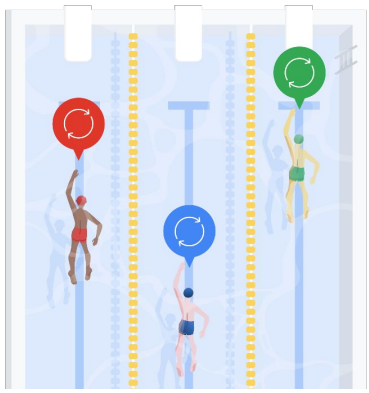
Google Cloud

Team-based sequences require each team to have its own dedicated cluster and team scope must be in a separate fleet.

GKE upgrades cluster control planes first, followed by nodes, while respecting maintenance windows and exclusions.

Proprietary + Confidential

Soak time is the delay between upgrades



Group A | Group B | Group C

- Can be used to monitor and validate workloads before proceeding to the next stage in a rollout sequence.
- You can override soak times and use maintenance windows and exclusions to manage individual cluster upgrades.
- For automatic upgrades, clusters must have the same upgrade target. Version discrepancies can be addressed by upgrading manually or removing clusters.

Google Cloud

Soak time is the delay between upgrades in different groups.

Soak time can be used to monitor and validate workloads before proceeding to the next stage in a rollout sequence.

For fine-grained control, you can override soak times and use maintenance windows and exclusions to manage individual cluster upgrades.

For automatic upgrades, all clusters in a rollout sequence must have the same upgrade target. Version discrepancies can be addressed by upgrading manually or removing clusters.

Proprietary + Confidential

Create a rollout sequence

- Google Cloud console
- Terraform

01 Determine the order in which groups will receive the update.

02 Set the soak time for each group.

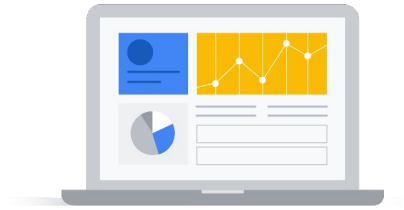
You can use gcloud commands or Terraform to create and manage rollout sequences.

When creating a sequence, determine the order in which groups will receive the update, and set the soak time for each group.

Proprietary + Confidential

Check the status of a rollout sequence

- **INELIGIBLE:** One or more clusters in the group are ineligible for the upgrade.
- **PENDING:** The upgrade hasn't started on any of the clusters in the group.
- **IN_PROGRESS:** The upgrade has started on at least one cluster but hasn't finished on all clusters.
- **SOAKING:** The upgrade has finished on all clusters and hasn't finished soaking.
- **FORCED_SOAKING:** The upgrade took more than the maximum upgrade time (30 days) and therefore it was forced to enter the soaking phase. The upgrade can continue in the clusters.
- **COMPLETE:** The upgrade has successfully completed its soak time and is ready for the next group.



gcloud commands

GKE Hub API

Google Cloud

To check the status of the sequence and individual clusters, you can use gcloud commands or the GKE Hub API. Progress on fleet-based rollouts can be monitored visually in the Google Cloud console.

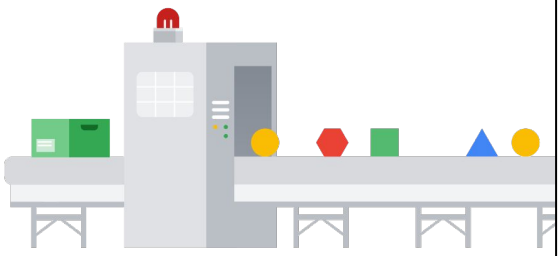
Returned status updates include:

- INELIGIBLE: One or more clusters in the group are ineligible for the upgrade.
- PENDING: The upgrade hasn't started on any of the clusters in the group.
- IN_PROGRESS: The upgrade has started on at least one cluster but hasn't finished on all clusters.
- SOAKING: The upgrade has finished on all clusters and hasn't finished soaking.
- FORCED_SOAKING: The upgrade took more than the maximum upgrade time (30 days) and therefore it was forced to enter the soaking phase. The upgrade can continue in the clusters.
- COMPLETE: The upgrade has successfully completed its soak time and is ready for the next group.

Proprietary + Confidential

Manage a rollout sequence

- Change the soak time for a group (default or version-specific override).
- Change the sequence order by updating upstream groups.
- Use maintenance exclusions or adjust the soak time to delay group completion.
- Switch between fleet-based and team-based rollout sequences.
- Delete a sequence by removing upstream associations.



For clusters outside Google Cloud, the **Connect Agent** establishes secure communication with the fleet host project, enabling seamless management.

Google Cloud

To manage a rollout sequence, you can:

- Change the soak time for a group (default or version-specific override).
- Change the sequence order by updating upstream groups.
- Use maintenance exclusions or adjust the soak time to delay group completion.
- Switch between fleet-based and team-based rollout sequences.
- Delete a sequence by removing upstream associations.

For clusters outside Google Cloud, the **Connect Agent** establishes secure communication with the fleet host project, enabling seamless management.

If trust cannot be achieved, two separate meshes could be connected, but this would result in additional management overhead.